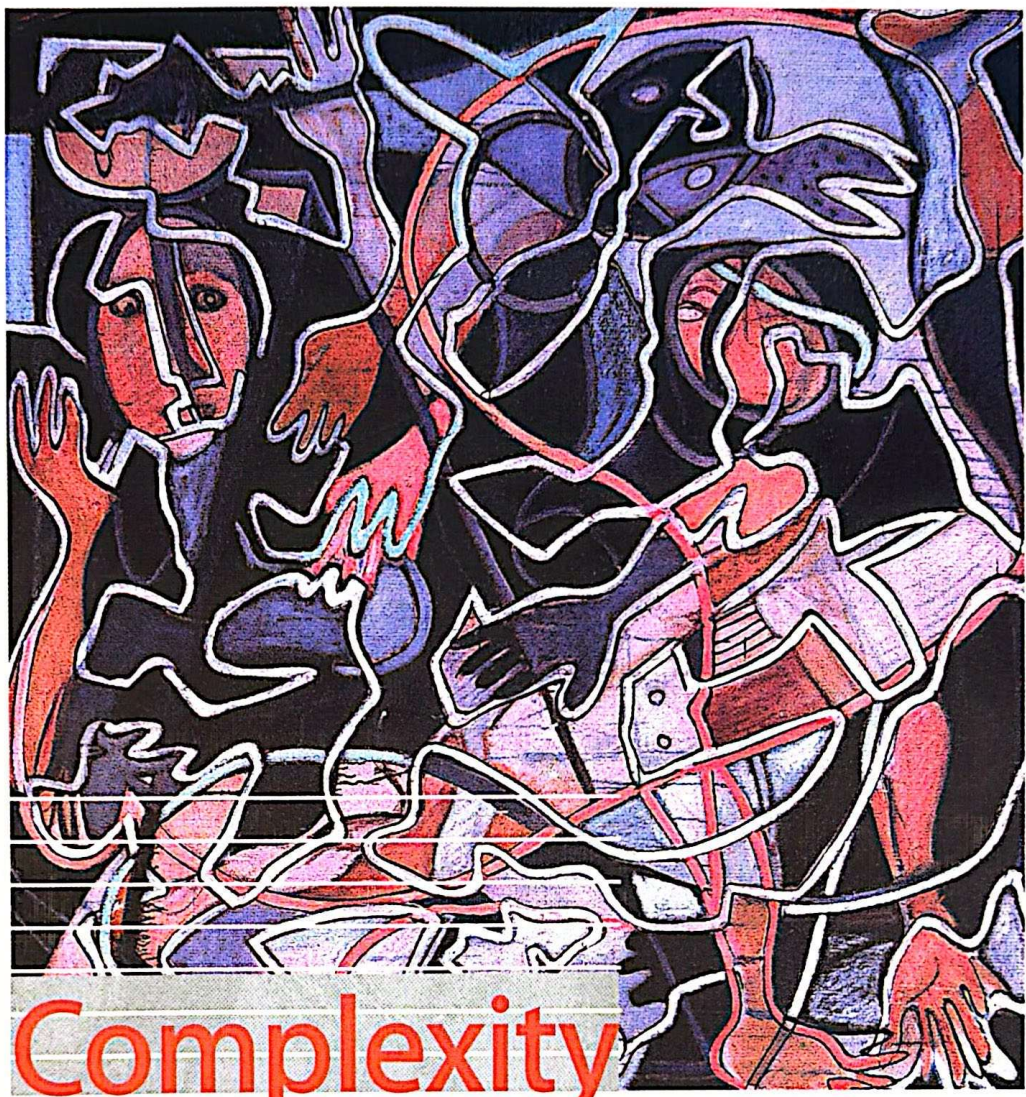


G. Ausiello
P. Crescenzi
G. Gambosi
V. Kann
A. Marchetti-
Spaccamela
M. Protasi



Complexity and Approximation

Combinatorial
Optimization Problems
and Their
Approximability Properties

Universitat Autònoma de Barcelona
Servei de Biblioteques



1500764924



Springer



Contents

1	The Complexity of Optimization Problems	1
1.1	Analysis of algorithms and complexity of problems	2
1.1.1	Complexity analysis of computer programs	3
1.1.2	Upper and lower bounds on the complexity of problems	8
1.2	Complexity classes of decision problems	9
1.2.1	The class NP	12
1.3	Reducibility among problems	17
1.3.1	Karp and Turing reducibility	17
1.3.2	NP-complete problems	21
1.4	Complexity of optimization problems	22
1.4.1	Optimization problems	22
1.4.2	PO and NPO problems	26
1.4.3	NP-hard optimization problems	29
1.4.4	Optimization problems and evaluation problems	31
1.5	Exercises	33
1.6	Bibliographical notes	36
2	Design Techniques for Approximation Algorithms	39
2.1	The greedy method	40
2.1.1	Greedy algorithm for the knapsack problem	41
2.1.2	Greedy algorithm for the independent set problem	43
2.1.3	Greedy algorithm for the salesperson problem	47

Table of contents

2.2	Sequential algorithms for partitioning problems	50
2.2.1	Scheduling jobs on identical machines	51
2.2.2	Sequential algorithms for bin packing	53
2.2.3	Sequential algorithms for the graph coloring problem	58
2.3	Local search	61
2.3.1	Local search algorithms for the cut problem	62
2.3.2	Local search algorithms for the salesperson problem	64
2.4	Linear programming based algorithms	65
2.4.1	Rounding the solution of a linear program	66
2.4.2	Primal-dual algorithms	67
2.5	Dynamic programming	69
2.6	Randomized algorithms	74
2.7	Approaches to the approximate solution of problems	76
2.7.1	Performance guarantee: chapters 3 and 4	76
2.7.2	Randomized algorithms: chapter 5	77
2.7.3	Probabilistic analysis: chapter 9	77
2.7.4	Heuristics: chapter 10	78
2.7.5	Final remarks	79
2.8	Exercises	79
2.9	Bibliographical notes	83
3	Approximation Classes	87
3.1	Approximate solutions with guaranteed performance	88
3.1.1	Absolute approximation	88
3.1.2	Relative approximation	90
3.1.3	Approximability and non-approximability of TSP	94
3.1.4	Limits to approximability: The gap technique	100
3.2	Polynomial-time approximation schemes	102
3.2.1	The class PTAS	105
3.2.2	APX versus PTAS	110
3.3	Fully polynomial-time approximation schemes	111
3.3.1	The class FPTAS	111
3.3.2	The variable partitioning technique	112
3.3.3	Negative results for the class FPTAS	113
3.3.4	Strong NP-completeness and pseudo-polynomiality	114
3.4	Exercises	116
3.5	Bibliographical notes	119
4	Input-Dependent and Asymptotic Approximation	123
4.1	Between APX and NPO	124
4.1.1	Approximating the set cover problem	124
4.1.2	Approximating the graph coloring problem	127

4.1.3	Approximating the minimum multi-cut problem . . .	129
4.2	Between APX and PTAS	139
4.2.1	Approximating the edge coloring problem	139
4.2.2	Approximating the bin packing problem	143
4.3	Exercises	148
4.4	Bibliographical notes	150
5	Approximation through Randomization	153
5.1	Randomized algorithms for weighted vertex cover	154
5.2	Randomized algorithms for weighted satisfiability	157
5.2.1	A new randomized approximation algorithm	157
5.2.2	A 4/3-approximation randomized algorithm	160
5.3	Algorithms based on semidefinite programming	162
5.3.1	Improved algorithms for weighted 2-satisfiability	167
5.4	The method of the conditional probabilities	168
5.5	Exercises	171
5.6	Bibliographical notes	173
6	NP, PCP and Non-approximability Results	175
6.1	Formal complexity theory	175
6.1.1	Turing machines	175
6.1.2	Deterministic Turing machines	178
6.1.3	Nondeterministic Turing machines	180
6.1.4	Time and space complexity	181
6.1.5	NP-completeness and Cook-Levin theorem	184
6.2	Oracles	188
6.2.1	Oracle Turing machines	189
6.3	The PCP model	190
6.3.1	Membership proofs	190
6.3.2	Probabilistic Turing machines	191
6.3.3	Verifiers and PCP	193
6.3.4	A different view of NP	194
6.4	Using PCP to prove non-approximability results	195
6.4.1	The maximum satisfiability problem	196
6.4.2	The maximum clique problem	198
6.5	Exercises	200
6.6	Bibliographical notes	204
7	The PCP theorem	207
7.1	Transparent long proofs	208
7.1.1	Linear functions	210
7.1.2	Arithmetization	214

Table of contents

7.1.3	The first PCP result	218
7.2	Almost transparent short proofs	221
7.2.1	Low-degree polynomials	222
7.2.2	Arithmetization (revisited)	231
7.2.3	The second PCP result	238
7.3	The final proof	239
7.3.1	Normal form verifiers	241
7.3.2	The composition lemma	245
7.4	Exercises	248
7.5	Bibliographical notes	249
8	Approximation Preserving Reductions	253
8.1	The World of NPO Problems	254
8.2	AP-reducibility	256
8.2.1	Complete problems	261
8.3	NPO-completeness	261
8.3.1	Other NPO-complete problems	265
8.3.2	Completeness in exp-APX	265
8.4	APX-completeness	266
8.4.1	Other APX-complete problems	270
8.5	Exercises	281
8.6	Bibliographical notes	283
9	Probabilistic analysis of approximation algorithms	287
9.1	Introduction	288
9.1.1	Goals of probabilistic analysis	289
9.2	Techniques for the probabilistic analysis of algorithms	291
9.2.1	Conditioning in the analysis of algorithms	291
9.2.2	The first and the second moment methods	293
9.2.3	Convergence of random variables	294
9.3	Probabilistic analysis and multiprocessor scheduling	296
9.4	Probabilistic analysis and bin packing	298
9.5	Probabilistic analysis and maximum clique	302
9.6	Probabilistic analysis and graph coloring	311
9.7	Probabilistic analysis and Euclidean TSP	312
9.8	Exercises	316
9.9	Bibliographical notes	318
10	Heuristic methods	321
10.1	Types of heuristics	322
10.2	Construction heuristics	325
10.3	Local search heuristics	329

10.3.1	Fixed-depth local search heuristics	330
10.3.2	Variable-depth local search heuristics	336
10.4	Heuristics based on local search	341
10.4.1	Simulated annealing	341
10.4.2	Genetic algorithms	344
10.4.3	Tabu search	347
10.5	Exercises	349
10.6	Bibliographical notes	350
A	Mathematical preliminaries	353
A.1	Sets	353
A.1.1	Sequences, tuples and matrices	354
A.2	Functions and relations	355
A.3	Graphs	356
A.4	Strings and languages	357
A.5	Boolean logic	357
A.6	Probability	358
A.6.1	Random variables	359
A.7	Linear programming	361
A.8	Two famous formulas	365
B	A List of NP Optimization Problems	367
	Bibliography	471
	Index	515

Chapter

1

The Complexity of Optimization Problems

IN INTRODUCTORY computer programming courses we learn that computers are used to execute algorithms for the solution of *problems*. Actually, the problems we want to solve by computer may have quite varying characteristics. In general, we are able to express our problem in terms of some relation $P \subseteq I \times S$, where I is the set of *problem instances* and S is the set of *problem solutions*. As an alternative view, we can also consider a predicate $p(x, y)$ which is true if and only if $(x, y) \in P$. If we want to analyze the properties of the computations to be performed, it is necessary to consider the characteristics of the sets I , S and of the relation P (or of the predicate p) more closely.

In some cases, we just want to determine if an instance $x \in I$ satisfies a given condition, i.e., whether $\pi(x)$ is verified, where π is a specified (unary) predicate. This happens, for example, when we want to check if a program is syntactically correct, or a certain number is prime, or when we use a theorem prover to decide if a logical formula is a theorem in a given theory. In all these cases, relation P reduces to a function $f : I \mapsto S$, where S is the binary set $S = \{\text{YES}, \text{NO}\}$ (or $S = \{0, 1\}$), and we denote our problem as a *decision* (or *recognition*) *problem*. We may also consider *search* problems, where, for any instance $x \in I$, a solution $y \in S$ has to be returned such that $(x, y) \in P$ is verified. This includes such problems as, for example, finding a path in a graph between two given nodes or determining the (unique) factorization of an integer. In other cases, given an instance $x \in I$, we are interested in finding the “best” solution y^* (according to some measure) among all solutions $y \in S$ such that $(x, y) \in P$ is verified. This

Universitat Autònoma de Barcelona

Biblioteca de Ciències
i d'Enginyeries

Fons Bibliogràfic de Matemàtiques

is the case when, given a point q and a set of points Q in the plane, we want to determine the point $q' \in Q$ which is nearest to q or when, given a weighted graph, we want to find a Hamiltonian cycle, if any, of minimum cost. Problems of this kind are called *optimization problems* and have occurred frequently in most human activities, since the beginning of the history of mathematics.

The aim of this book is to discuss how and under what conditions optimization problems which are computationally hard to solve can be efficiently approached by means of algorithms which only return “good” (and possibly not the best) solutions. In order to approach this topic, we first have to recall how the efficiency of the algorithms and the computational complexity of problems are measured. The goal of this chapter is, indeed, to introduce the reader to the basic concepts related to the complexity of optimization problems. Since the whole setting of complexity theory is built up in terms of decision problems, we will first discuss these concepts by showing the main results concerning decision problems. In particular, we will introduce the two complexity classes P and NP, whose properties have been deeply investigated in the last decades, and we will briefly discuss their relationship. Subsequently, we will shift our attention to the optimization problem context and we will show how the previously introduced concepts can be adapted to deal with this new framework.

1.1 Analysis of algorithms and complexity of problems

THE MOST direct way to define the efficiency of an algorithm would be to consider how much time (or memory), for any instance, the algorithm takes to run and output a result on a given machine. Such a point of view depends on the structural and technological characteristics of the machines and of their system software. It is possible to see that, as a matter of fact, the cost of the same algorithm on two different machines will differ only by no more than a multiplicative constant, thus making cost evaluations for a certain machine significant also in different computing environments.

As a consequence, the algorithms presented throughout the book will be usually written in a Pascal-like language and will be hypothetically run on an abstract Pascal machine, composed by a control and processing unit, able to execute Pascal statements, and a set of memory locations identified by all variable and constant identifiers defined in the algorithm. Unless otherwise specified, all statements concerning the efficiency of algorithms and the complexity of problem solutions will refer to such a computation model.

1.1.1 Complexity analysis of computer programs

The simplest way to measure the running time of a program in the model chosen above is the **uniform cost measure** and consists in determining the overall number of instructions executed by the algorithm before halting.

This approach to computing the execution cost of a program is natural, but, since it assumes that any operation can be executed in constant time on operands of any size (even arbitrarily large), it may lead to serious anomalies if we consider the possibility that arbitrarily large values can be represented in memory locations. Thus, in practice, the uniform cost model may be applied in all cases where it is implicitly assumed that all memory locations have the same given size and the values involved in any execution of the algorithm are not greater than that size, i.e., any value to be represented during an execution of the algorithm can be stored in a memory location. In all other cases (for example, where a bound on the size of the values involved cannot be assumed) a different cost model, known as **logarithmic cost model** should be used.

The logarithmic cost model is obtained by assigning to all instructions a cost which is a function of the number of bits (or, equivalently for positive integers, of the logarithm) of all values involved. In particular, basic instructions such as additions, comparisons, and assignments are assumed to have cost proportional to the number of bits of the operands, while a cost $O(n \log n)$ for multiplying or dividing two n -bit integers may be adopted.

For example, in the execution of an assignment instruction such as $a := b + 5$, we consider the execution cost equal to $\log |b| + \log 5$. Notice that we did not specify the base of the logarithm, since logarithms in different bases differ only by a multiplicative constant.

Such a cost model avoids the anomalies mentioned above and corresponds to determining the number of operations required to perform arithmetical operations with arbitrary precision on a real computer.

Both approaches can also be applied to evaluate the execution cost in terms of the amount of memory (space) used by the algorithm. In this framework, the uniform cost model will take into account the overall number of distinct memory locations accessed by the algorithm during the computation. On the other hand, according to the logarithmic cost model it should be necessary to consider the number of bits of the maximum value contained in such locations during the computation.

Consider Program 1.1 for computing $r = x^y$, where $x, y \in \mathbb{N}$. In the uniform cost model, this algorithm has time cost $2 + 3y$, while in the logarithmic cost model, it presents a time cost bounded by the expression $ay \log y + by^2 \log x (\log y + \log \log x) + c$, where a, b, c are suitable constants (see Exercise 1.1). Concern-

◀ Example 1.1

Program 1.1: Exponentiation

```

input Nonnegative integers  $x, y$ ;
output Nonnegative integer  $r = x^y$ ;
begin
   $r := 1$ ;
  while  $y \neq 0$  do
    begin
       $r := r * x$ ;
       $y := y - 1$ 
    end;
  return  $r$ 
end.

```

ing the amount of memory used, the resulting uniform cost is 3 (the variables x , y , r), while the logarithmic cost is $\log y + (y + 1) \log x$.

In the following, we will refer to the uniform cost model in all (time and space) complexity evaluations performed. This is justified by the fact that the numbers used by the algorithms described in this book will be sufficiently small with respect to the length of their inputs. Before going into greater detail, we need to make clear some aspects of cost analysis that may help us to make the evaluation easier and more expressive and to introduce a suitable notation.

Worst case analysis. [The first aspect we have to make clear is that, for most applications, at least as a first step of the analysis, we are not interested in determining the precise running time or space of an algorithm for all particular values of the input. Our concern is rather more in determining the **behavior** of these execution costs **as the input size grows**.] However, instances of the same size may anyway result in extremely different execution costs. It is well known, for example, that sorting algorithms may have a different behavior, from the point of view of the running time, if they have to sort an array whose elements are randomly chosen or an array of partially sorted items. For such reasons, [in order to specify the performance of an algorithm on inputs of size n , we determine the cost of applying the algorithm on the **worst case** instance of that size, that is on the instance with highest execution cost.]

In contrast with worst case analysis, in several situations a different approach may be taken (known as **average case** analysis), consisting of determining the average cost of running the algorithm on all instances of size n , assuming that such instances occur with a specified (usually uniform) probability distribution.

Worst case analysis is widely used since it provides us with the certainty that, in any case, the given algorithm will perform its task within the established time bound. Besides, it is often easier to obtain, while the average case analysis may require complex mathematical calculations and, moreover, has validity limited by the probabilistic assumptions that have been made on the input distribution.

Input size. As stated above, we are interested in expressing the execution costs as a (growing) function of the size of the instance of the problem to be solved. But, how to measure that size? In Example 1.1 the running time of the algorithm was provided as a function of the input values x, y . Even if this seemed a reasonable choice in the case of the computation of an integer function, conventionally, a different *input size measure* is adopted: the *length* or *size of the input*, that is the number of digits (possibly bits) needed to present the specific instance of the problem. This conventional choice is motivated by the fact that problems may be defined over quite heterogeneous data: integers or sequences of integers, sets, graphs, geometric objects, arrays and matrices, etc. In all cases, in order to be submitted as input to a computer algorithm, the data will have to be presented in the form of a character string over a suitable finite alphabet (e.g., the binary alphabet $\{0, 1\}$). It is therefore natural to adopt the length of such a string as a universal input size measure good for all kinds of problems.

Thus we assume there exists an *encoding scheme* which is used to describe any problem instance in a string of characters over some alphabet. Even if different encoding schemes usually result in different strings (and input sizes) for the same instance, it is important to observe that for a wide class of *natural* encoding schemes (i.e., encoding schemes that do not introduce an artificially redundant description) the input sizes of the same instance are not too different from each other. This is expressed by saying that for any pair of natural encoding schemes e_1, e_2 and for any problem instance x , the resulting strings are polynomially related, that is there exist two polynomials p_1, p_2 such that $|e_1(x)| \leq p_1(|e_2(x)|)$ and $|e_2(x)| \leq p_2(|e_1(x)|)$, where $|e_i(x)|$ denotes the length of string $e_i(x)$, for $i = 1, 2$.

In the following, for any problem instance x , we will denote with the same symbol x the string resulting by any natural encoding of the instance itself. In general, we will denote with the same symbol both an object and its encoding under any encoding scheme.

Let us consider the problem of determining if an integer $x \in \mathbb{Z}^+$ is a prime number. A trivial algorithm which tries all possible divisors d , $1 \leq d \leq \sqrt{x}$, can perform (if x is prime) a number of steps proportional to $\sqrt{x}$. If we use the natural encoding scheme which represents integer x as a binary string of length $n = |x| \approx \log x$, the

◀ Example 1.2

execution cost is instead proportional to $2^{n/2}$. Even if the two values are equal, the second one gives greater evidence of the complexity of the algorithm, which grows exponentially with the size of the input. Actually, at the current state of knowledge, it is not yet known whether any algorithm for primality testing with a polynomial number of steps exists. Notice that using a unary base encoding (an unnatural scheme) would result in an evaluation of the execution cost as the square root of the instance size.

Asymptotic analysis of algorithms. If we go back to the analysis of the running time of Program 1.1, we realize the following two facts. First, the expressions contain constants, which represent both the cost of some instructions (such as during the initialization phase of an algorithm) whose execution does not depend on the particular instance and the fact that single instructions, on real machines, present execution costs which are not unitary but depend on technological characteristics of the machine (and system software) and would have been extremely difficult to determine precisely. Second, the expressions themselves are upper bounds and not precise evaluations of the computation costs.

For the above two reasons, by making use of the standard *O*-notation (see Appendix A), we may as well describe the execution costs of the algorithm by saying that its execution time **asymptotically grows** no more than $O(y)$ in the uniform cost model and $O(y^2 \log x (\log y + \log \log x))$ in the logarithmic cost model.

More generally, let us see how the notation used to express the asymptotic behavior of functions can be used to describe the running time (or space) of an algorithm.

Let us denote as $\hat{t}_{\mathcal{A}}(x)$ the **running time** of algorithm $\mathcal{A}$ on input x . The **worst case running time** of $\mathcal{A}$ is then given by

$$t_{\mathcal{A}}(n) = \max\{\hat{t}_{\mathcal{A}}(x) \mid \forall x: |x| \leq n\}.$$

Similarly, let us denote as $\hat{s}_{\mathcal{A}}(x)$ the **running space** of algorithm $\mathcal{A}$ on input x . The **worst case running space** of $\mathcal{A}$ is then given by

$$s_{\mathcal{A}}(n) = \max\{\hat{s}_{\mathcal{A}}(x) \mid \forall x: |x| \leq n\}.$$

Definition 1.1 ▶ We say that algorithm $\mathcal{A}$
Algorithm complexity bounds

1. has **complexity (upper bound)** $O(g(n))$ if $t_{\mathcal{A}}(n)$ is $O(g(n))$;
2. has **complexity (lower bound)** $\Omega(g(n))$ if $t_{\mathcal{A}}(n)$ is $\Omega(g(n))$;
3. has **complexity (exactly)** $\Theta(g(n))$ if $t_{\mathcal{A}}(n)$ is $\Theta(g(n))$.

Similar definitions can be introduced for the space complexity.

Section 1.1

ANALYSIS OF ALGORITHMS AND COMPLEXITY OF PROBLEMS

By using the preceding notation, if an algorithm performs a number of steps bounded by $an^2 + b \log n$ to process an input of size n , we say that its complexity is $O(n^2)$. If, moreover, we are able to prove that for any n sufficiently large there exists an input instance of size n on which the algorithm performs at least cn^2 steps, we say that its complexity is also $\Omega(n^2)$. In such a case we also say that the complexity of the algorithm is $\Theta(n^2)$.

Let us again consider the analysis of Program 1.1 in Example 1.1. As we have already seen, the running time of this algorithm under logarithmic cost measure is $ay \log y + by^2 \log x (\log y + \log \log x) + c$. As noticed above, the term $by^2 \log x (\log y + \log \log x)$ dominates all others. Let $n = |x| + |y|$ be the overall input size: a proper expression for the asymptotic cost analysis as a function of the input size would be $O(n^2 2^{2n})$.

◀ Example 1.3

The typical procedure to perform a complexity analysis of an algorithm requires deciding, case by case, on the following issues:

1. How to measure the execution cost. Usually, in the case of evaluating the running time, this is done by determining a *dominant* operation, that is an operation which is executed at least as often as any other operation in the algorithm, thus characterizing the asymptotic complexity of the algorithm. More formally, if the asymptotic complexity of the algorithm is $\Theta(g(n))$, a dominant operation is any operation whose contribution to the cost is $\Theta(g(n))$.

Typically, in a sorting algorithm a dominant operation is the comparison of two elements, in matrix multiplication a dominant operation may be assumed to be the multiplication of elements, etc.

2. Which cost measure we want to adopt. As said above, the logarithmic cost model is more precise, but the simpler uniform cost model can be applied if we make the assumption that all values involved in the algorithm execution are upper bounded by some value (usually some function of the input size).
3. How to measure the input size, that is what characteristic parameter of the input instance is the one whose growth towards infinity determines the asymptotic growth of the computation cost. In this case, typical examples are the number of graph nodes and edges in graph algorithms, the number of rows and columns of matrices in algebraic algorithms, etc.

Let us consider a simple version of the well known *Insertion Sort* algorithm (see Program 1.2). Suppose that an array of size n is given and that we want to sort

◀ Example 1.4

Program 1.2: Insertion Sort

```

input Array  $A[1, \dots, n]$  of integers;
output Array  $A$  sorted in increasing order;
begin
  for  $i := 1$  to  $n - 1$  do
    begin
       $j := i + 1$ ;
      while  $j \geq 2$  and  $A[j] < A[j - 1]$  do
        begin
          swap  $A[j]$  and  $A[j - 1]$ ;
           $j := j - 1$ 
        end
      end
    end;
  return  $A$ 
end.

```

it in increasing order. Without loss of generality let us assume that the range of the elements is the finite interval $\{1, \dots, M\}$, where M is a constant. Here the input size is characterized by the number n of elements in the array. In the uniform model we may simply take into consideration comparison operations. Since the algorithm contains two loops each of which is repeated at most n times, the running time is clearly bounded by $O(n^2)$. On the other side, it is not difficult to observe that the number of comparisons performed by the algorithm is indeed $n(n - 1)/2$ in the worst case, that is when the array is ordered in decreasing order. Hence, the algorithm also has a lower bound $\Omega(n^2)$ on the running time.

Regarding the running space of the algorithm, it is immediate to see that an $\Omega(n)$ lower bound holds, due to the need for accessing all values in the input instance. The algorithm accesses such values plus a constant number of additional locations, thus resulting in a $O(n)$ upper bound.

1.1.2 Upper and lower bounds on the complexity of problems

Suppose we have to solve a problem and we have an algorithm whose time complexity is $O(g(n))$. According to the following definition, we say that $O(g(n))$ is an upper bound to the time complexity of the given problem.

Definition 1.2 ▶ *Given a problem $\mathcal{P}$ we say that the **time complexity upper bound** of $\mathcal{P}$ is $O(g(n))$ if there exists an algorithm for $\mathcal{P}$ whose running time is $O(g(n))$.*

Problem complexity upper bound

In other words, the complexity upper bound for a problem provides information on the amount of time which is asymptotically sufficient to

Section 1.2

COMPLEXITY CLASSES OF DECISION PROBLEMS

solve the problem. Knowing the upper bound does not mean that a precise knowledge of the complexity of the problem is available. In fact, other more efficient algorithms with smaller running times may exist, and we may just be unaware of their existence.

A more precise characterization of the time complexity of a problem is achieved when we are also able to find a time complexity lower bound, that is we are able to establish how much time is anyhow needed (asymptotically) to solve the problem, no matter what algorithms are used. Knowing a lower bound for a problem provides important information on the intrinsic difficulty of solving the problem, especially in the case that the lower bound is sufficiently close to (or even coincides with) the upper bound.

Given a problem $\mathcal{P}$ we say that the **time complexity lower bound** of $\mathcal{P}$ is $\Omega(g(n))$ if any algorithm for $\mathcal{P}$ has a running time $\Omega(g(n))$, and that the **time complexity** of $\mathcal{P}$ is $\Theta(g(n))$ if its **upper bound** is $O(g(n))$ and its lower bound is $\Omega(g(n))$.

◀ Definition 1.3
Problem complexity bounds

← Rate, no?

Establishing a complexity lower bound for a problem is a hard task, since it requires stating (and proving) a property that has to hold for all (known and unknown) algorithms that solve the problem.

In the case of sorting, by means of an information theoretic argument, it can be proved that, in terms of comparisons, the time complexity lower bound is $\Omega(n \log n)$, that is any algorithm for sorting that uses only comparisons requires $\Omega(n \log n)$ comparisons. Since the time complexity upper bound for sorting is also $O(n \log n)$, this means that, in terms of comparisons, the time complexity of sorting is exactly determined to be $\Theta(n \log n)$.

Unfortunately, such a precise characterization is not achieved in all cases. For most problems of great practical interest (the satisfiability of a Boolean formula in the propositional calculus, the multiplication of two matrices, the decomposition of a number into prime factors, etc.) the time complexity has not presently been precisely determined. In particular, this is the case for all optimization problems which form the subject of this book. Despite this difficulty, a classification of (both decision and optimization) problems according to their complexity can be established in formal terms. Before examining how the complexity of optimization problems can be defined, let us recall some basic notions about the complexity of decision problems.

1.2 Complexity classes of decision problems

A PROBLEM $\mathcal{P}$ is called a *decision problem* if the set $I_{\mathcal{P}}$ of all instances of $\mathcal{P}$ is partitioned into a set $Y_{\mathcal{P}}$ of *positive instances* and a set $N_{\mathcal{P}}$ of